

AI-Based Mutual Fund Analyzer & Recommender

Submitted in partial fulfillment of the requirements for the degree of

Master of Data Science (Online)

by

Rahul Bhagat

24EMDT1062

Under the guidance

of

Prof. RAJA DAS



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

June 2026

VIT Online Learning Program

DECLARATION

I, **Rahul Bhagat**, with register number **24EMDT1062** hereby declare that the project report entitled “**AI-Based Mutual Fund Analyzer & Recommender**” submitted by me to M.Sc., Data Science VIT Online learning program, Vellore, in partial fulfilment of the requirement for the award of the degree of **Master of Data Science** is a Bonafide work carried out by me under the supervision of **Prof. RAJA DAS**, Associate Professor, Department of Mathematics, School of Advanced Sciences (SAS), Vellore Institute of Technology, Vellore - 632 014.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other Institute or University.

Place: VIT VELLORE

Date: 30- JUN - 2026

Rahul Bhagat

**Signature of the
Candidate**

CERTIFICATE

This is to certify that the project work entitled “**AI-Based Mutual Fund Analyzer & Recommender**” submitted by **Rahul Bhagat** with registration number **24EMDT1062**, to VIT Vellore, in partial fulfilment of the requirement for the award of the degree of Master of Business Administration, is a bona fide work carried out by him/her under my supervision. The project fulfils the requirements as per VIT Vellore regulations and, in my opinion, meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this Institute or any other Institute or University.

Place: VIT VELLORE

Date: 30-JUN- 2026

Prof. Dr. Raja Das

Guide Name & Signature

Examiner 1: HOD Online M.Sc. DS

Examiner 2: Director, VITOL

ACKNOWLEDGEMENT

At the outset, I thank the Almighty God for His blessings for granting me the knowledge and right aptitude to successfully complete my project work.

I would like to express my special gratitude and thanks to my guide **Prof. Dr. Raja Das, Associate Professor**, Department of Mathematics, School of Advanced Sciences (SAS), whose esteemed guidance and immense support encouraged me to complete the project successfully.

My sincere thanks to Honorable Chancellor, Dr. G. VISWANATHAN; esteemed Vice-Presidents; respected Vice Chancellor, Dr. V. S. KANCHANA BHAASKARAN of this prestigious VIT, Vellore, for providing me with an excellent world-class academic environment and facilities for pursuing my online MBA Program.

My sincere gratitude lies to the Director, **Dr. RHYMEND UTHARIARAJ** VITOL, and the Head of the Department, online M.Sc. Data Science, VITOL, **Prof. Sri Rama Vara Prasad Bhuvanagiri**, for providing me with an opportunity to do my project work at VIT, Vellore.

I also thank all the faculty members of the VITOL, Department of Mathematics, and the faculty of other Departments of the VIT, as well as the non-teaching staff, for giving me the courage and strength that I needed to achieve my goals.

My special thanks to my friends for their timely help and suggestions rendered for the successful completion of this project.

This acknowledgement would be incomplete without expressing my whole-hearted thanks to my parents for their continuous support and guidance in all walks of my life.

Rahul Bhagat

Project Title	AI-Based Mutual Fund Analyzer & Recommender
Name of the learner	Rahul Bhagat - 24EMDT1062
Name of the Institution	Vellore Institute of Technology (VIT)
Project Guide	Prof. Dr. Raja Das
Program	Online M.Sc. Data Science
Academic Year	2024-26

ABSTRACT

This project presents an AI-based Mutual Fund Analyzer and Recommender System designed to help investors explore, compare, and select suitable mutual fund schemes using data-driven insights. The platform processes large-scale NAV data from Indian mutual funds to generate financial features such as returns, volatility, risk measures, momentum indicators, and trend signals.

The solution includes a RESTful FastAPI backend, an interactive Streamlit dashboard, and automated data pipelines for ingestion, preprocessing, feature engineering, ranking, and clustering. By combining financial analytics with machine learning techniques, the project reduces the complexity of mutual fund selection for retail investors, demonstrating how data science can build a practical, scalable, and user-friendly investment analysis tool.

Keywords: Mutual Fund Analysis, Financial Analytics, Machine Learning, Risk Assessment, Fast API, Streamlit , Airflow, Python

CONTENTS

TITLE	PAGE NO
DECLARATION	i
CERTIFICATE	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
CONTENTS	v
LIST OF TABLES	vi
LIST OF FIGURES	vii
CHAPTER – 1: INTRODUCTION	9-10
1.1 Background and Problem Statement	9
1.2 Motivation	9
1.3 Objectives and Scope	10
CHAPTER – 2: REVIEW OF LITERATURE	11-15
2.1 Mutual Fund Performance Evaluation and Risk Metrics	11
2.2 Financial Data Engineering and Feature Extraction Pipelines	12
2.3 Machine Learning Applications in Portfolio Clustering and Ranking	13
2.4 Interactive Financial Frameworks and Democratic Analytics	14
2.5 Summary and Identification of Literature Gap	15
CHAPTER – 3: SYSTEM ARCHITECTURE AND DESIGN	16-21

3.1 Introduction to System Architecture	16
3.2 Overall Architecture of the Proposed System	16
3.3 Module Design	17
3.4 Data Flow Design	18
3.5 Component Interaction	19
3.6 Backend Design (FastAPI REST Gateway)	19
3.7 Frontend/Dashboard Design (Streamlit Interface)	20
3.8 Machine Learning Design (Clustering & Utility Layout)	20
3.9 Database/Data Storage Design (PostgreSQL Schema)	21
3.10 Advantages of the Proposed Design	21
3.11 Conclusion	21
CHAPTER – 4: SYSTEM IMPLEMENTATION	22-27
4.1 Introduction to System Implementation	22
4.2 Software Environment and Tech Stack Setup (Docker Cluster)	22
4.3 Data Pipeline & Airflow Workflow Orchestration (6 DAGs)	23
4.4 Database Schema Realization	24
4.5 Backend API Endpoint Implementation	25
4.6 Frontend UI Page Architecture (8-Page Multi-App)	26
4.7 Algorithmic Logic Execution	27
4.8 Exception Handling and System Integrity	27
4.9 Conclusion	27
CHAPTER – 5: ALGORITHMIC LOGIC AND MACHINE LEARNING IMPLEMENTATION	28-33

5.1 Introduction to Algorithmic Framework	28
5.2 Mathematical Formulations for Feature Engineering (17 Metrics)	28
5.3 Unsupervised Learning via K-Means Clustering (\$K=4\$)	30
5.4 Utility Optimization-Based Recommendation	31
5.5 Algorithmic Execution Example (Step-by-Step Trace)	32
5.6 Conclusion	33
CHAPTER – 6: COMPARATIVE ANALYSIS, ADVANTAGES, AND LIMITATIONS	34-37
6.1 Introduction	34
6.2 Comparative Analysis with Existing Models and Approaches	34
6.3 Strategic Advantages and Project Benefits	35
6.4 System Limitations	36
6.5 Conclusion	37
REFERENCES	38

LIST OF TABLES

1	Table 1: 7-Layer Pipeline Architecture	29
---	--	----

LIST OF FIGURES

1	High-Level System Flow	16
2	UI Design	23

CHAPTER – 1

INTRODUCTION

1.1 Background and Problem Statement

The Indian mutual fund ecosystem has expanded rapidly, offering thousands of schemes across various asset classes and fund houses. However, navigating this extensive universe presents significant challenges for retail investors, who typically lack the specialized resources required for deep financial analysis. Due to the absence of accessible, objective platforms, many investors rely on fragmented advice from social media influencers or unverified online sources. This dependency frequently leads to sub-optimal investment choices, misaligned asset allocation, and exposure to risks that do not match the investor's actual financial goals or risk tolerance.

This challenge is further intensified by the sheer volume and complexity of the underlying market data. Historical Net Asset Value (NAV) records are vast, semi-structured, and prone to inconsistencies, making manual collection and cleaning highly inefficient. Furthermore, evaluating a fund's true performance requires analyzing more than just raw returns; it demands a comprehensive assessment of risk-adjusted performance, market volatility, and long-term trends. For an individual investor, calculating these metrics manually across thousands of competing funds is practically impossible, creating a distinct gap between raw market data and actionable insights.

Driven by these challenges, this project aims to simplify the mutual fund selection process by transforming raw financial data into a structured, user-friendly decision framework. The objective is to build a scalable system that automates data gathering, removes manual complexity, and systematically ranks schemes based on clear performance metrics and risk profiles. Ultimately, this work establishes a transparent, data-driven approach that reduces reliance on subjective external advice, bridging the gap between complex financial data and practical investment decision-making.

1.2 Motivation

The motivation behind this research is to democratize quantitative financial analysis by building a scalable, transparent, and reproducible end-to-end data pipeline. By automating the extraction, transformation, loading (ETL), and feature engineering processes, this project eliminates manual data wrangling and mitigates the noise inherent in raw market feeds.

To transform this processed data into actionable insights, the system leverages unsupervised machine learning (clustering) alongside rule-based scoring models. This hybrid approach systematically categorizes, ranks, and recommends mutual fund schemes based on distinct investor risk profiles.

Ultimately, this project serves a dual purpose:

Academic & Practical Paradigm: It establishes a robust, explainable framework for applied data science in financial markets, while simultaneously delivering a production-ready application. By deploying a high-performance RESTful FastAPI backend integrated with an intuitive Streamlit dashboard, this work provides retail investors with a professional-grade, data-driven utility to simplify mutual fund selection and optimize portfolio alignment.

1.3 Objectives and Scope

- Build an intelligent mutual fund analysis and recommendation system that helps users choose schemes based on performance, risk, and trend indicators.
- Automate the full data pipeline, including NAV data ingestion, cleaning, validation, and feature engineering.
- Apply machine learning techniques such as clustering and scoring to group, rank, and recommend mutual funds in a transparent way.
- Provide a web-based interface using FastAPI and Streamlit for scheme search, comparison, visualization, and risk analysis.
- Support retail investors with a data-driven tool focused on Indian mutual fund schemes, covering analytical decision-making from raw data to final recommendations.

CHAPTER – 2

REVIEW OF LITERATURE

The primary objective of this literature review is to evaluate the theoretical and empirical foundations that intersect financial analytics, data engineering, and machine learning within the context of investment decision-support systems. This chapter synthesizes existing research across four core dimensions: traditional mutual fund performance evaluation, financial data engineering and preprocessing paradigms, the application of machine learning in fund classification and ranking, and the evolution of accessible, web-driven financial architectures.

2.1 Mutual Fund Performance Evaluation and Risk Metrics

The foundational architecture of mutual fund analysis is deeply rooted in Modern Portfolio Theory (MPT), pioneered by Markowitz (1952), which established the trade-off between risk and return. Traditional literature heavily relies on mean-variance frameworks to assess portfolio performance. Sharpe (1966) introduced the reward-to-variability ratio, commonly known as the Sharpe Ratio, which evaluates excess return per unit of total volatility. However, asymmetric return distributions in mutual funds led to modifications; Sortino and van der Meer (1991) isolated downside deviation from total volatility to penalize only negative price fluctuations, creating the Sortino Ratio.

Subsequent empirical studies in financial analytics demonstrate that historical Net Asset Value (NAV) data, when viewed in isolation, fails to provide an accurate predictive or descriptive metric for future performance. As highlighted by systemic market studies, raw returns do not account for maximum drawdowns—representing the peak-to-trough decline during a specific period—nor do they capture structural trend signals or price momentum. Consequently, contemporary financial research emphasizes the necessity of incorporating multi-dimensional, time-series features to fully encapsulate a fund's risk-return profile, consistency, and structural behavior during market contractions.

2.2 Financial Data Engineering and Feature Extraction Pipelines

A critical prerequisite for robust financial modeling is the establishment of rigorous data preprocessing pipelines. Financial market data, particularly daily NAV records published by regulatory bodies like the Association of Mutual Funds in India (AMFI), is inherently semi-structured, heterogeneous, and prone to anomalies. Academic literature in financial data engineering establishes that raw market feeds frequently contain missing observations, reporting delays, and erroneous outliers caused by corporate actions or manual entry faults.

Prior research highlights that the mathematical fidelity of any downstream machine learning or statistical model is bound by the quality of the input data—a paradigm often referred to as "garbage in, garbage out." Consequently, automated data validation, outlier detection, and interpolation methods are paramount. Furthermore, feature engineering serves as the bridge between raw data and algorithmic insight. Transforming raw price series into rolling moving averages, annualized volatility metrics, cumulative trajectories, and momentum indicators like the Relative Strength Index (RSI) allows the underlying analytical models to extract latent temporal patterns and smooth out short-term market noise.

2.3 Machine Learning Applications in Portfolio Clustering and Ranking

In recent decades, the paradigm of investment analytics has shifted from purely linear statistical models to advanced machine learning methodologies. Within fund selection, unsupervised learning—specifically clustering—has emerged as a powerful mechanism for pattern recognition without predefined labels. Literature demonstrates that algorithms such as K-Means clustering are highly effective at partitioning broad financial universes into distinct, homogeneous peer groups based on shared volatility and return vectors. The mathematical simplicity, computational efficiency, and geometric interpretability of K-Means make it a benchmark in quantitative finance for identifying comparable assets.

Beyond unsupervised partitioning, multi-criteria decision-making (MCDM) and utility-based scoring models are widely documented as the standard for multi-factor ranking systems. In recommendation systems, combining diverse financial features (e.g., combining a fund's Sharpe ratio, assets under management, and momentum signals) into a singular, unified decision score requires robust normalization and scaling techniques. Literature indicates that such hybrid approaches—coupling algorithmic grouping with rule-based ranking functions—effectively translate high-dimensional financial characteristics into actionable, risk-segmented recommendations aligned with specific investor utility profiles.

2.4 Interactive Financial Frameworks and Democratic Analytics

The final domain of the literature explores the translation of complex quantitative models into accessible software architectures. Historically, advanced financial analytics tools were restricted to institutional environments via proprietary systems. However, recent computer science literature demonstrates a paradigm shift toward democratized, web-based analytical platforms.

The integration of high-performance, asynchronous backend frameworks like FastAPI with highly reactive, pythonic frontend interfaces like Streamlit has significantly lowered the barrier to deploying scalable data applications. Scholars in human-computer interaction (HCI) and financial technology emphasize that interactive visualizations, historical trend

dashboards, and transparent recommendation logic are critical for retail investors. By abstracting the underlying computational complexity behind RESTful APIs and clean interfaces, modern architectures provide explainable data insights, directly mitigating the information asymmetry that historically disadvantaged individual market participants.

CHAPTER - 3

SYSTEM ARCHITECTURE AND DESIGN

The architectural integrity of a data-intensive financial application dictates its stability, precision, and performance under variable computational loads. For an AI-based platform processing 18 years of historical quantitative market data across thousands of financial instruments, the architecture must balance complex mathematical computations with real-time consumer delivery. This chapter establishes the structural, component-level design of the “*AI-Mutual Fund Analyzer and Recommender System*”. The system is built upon a decoupling paradigm that segregates raw computational processing from the client-facing presentation layer. The architecture focuses on performance, scalability, fault tolerance, and mathematical reproducibility. converting noisy, semi-structured industry feeds into objective, structured investment strategies.

The system is built on a **5-layer decoupled architecture** designed to manage the lifecycle of mutual fund evaluation cleanly: Data Source, Data Pipeline (ETL), Machine Learning (ML) Engine, API Service Layer, and User Interface (UI) Layer.

The system operates as an asynchronous, containerized pipeline.

1. At the perimeter, the **Data Source Layer** fetches daily updates from the Association of Mutual Funds in India (AMFI).
2. This raw payload passes into the **Data Pipeline Layer**, which handles parsing, outlier cleaning, and feature engineering sequentially.
3. The processed information is saved in the **Database Layer (PostgreSQL 15)**.
4. The **Machine Learning Engine** pulls data from the database to group, score, and rank funds.
5. To keep performance high, the **API Service Layer (FastAPI)** pulls from an in-memory cache to expose data through endpoints.
6. Finally, the **UI Layer (Streamlit)** communicates exclusively with the API layer via HTTP, displaying the analytical charts to the end user without touching the underlying database or ML training models.

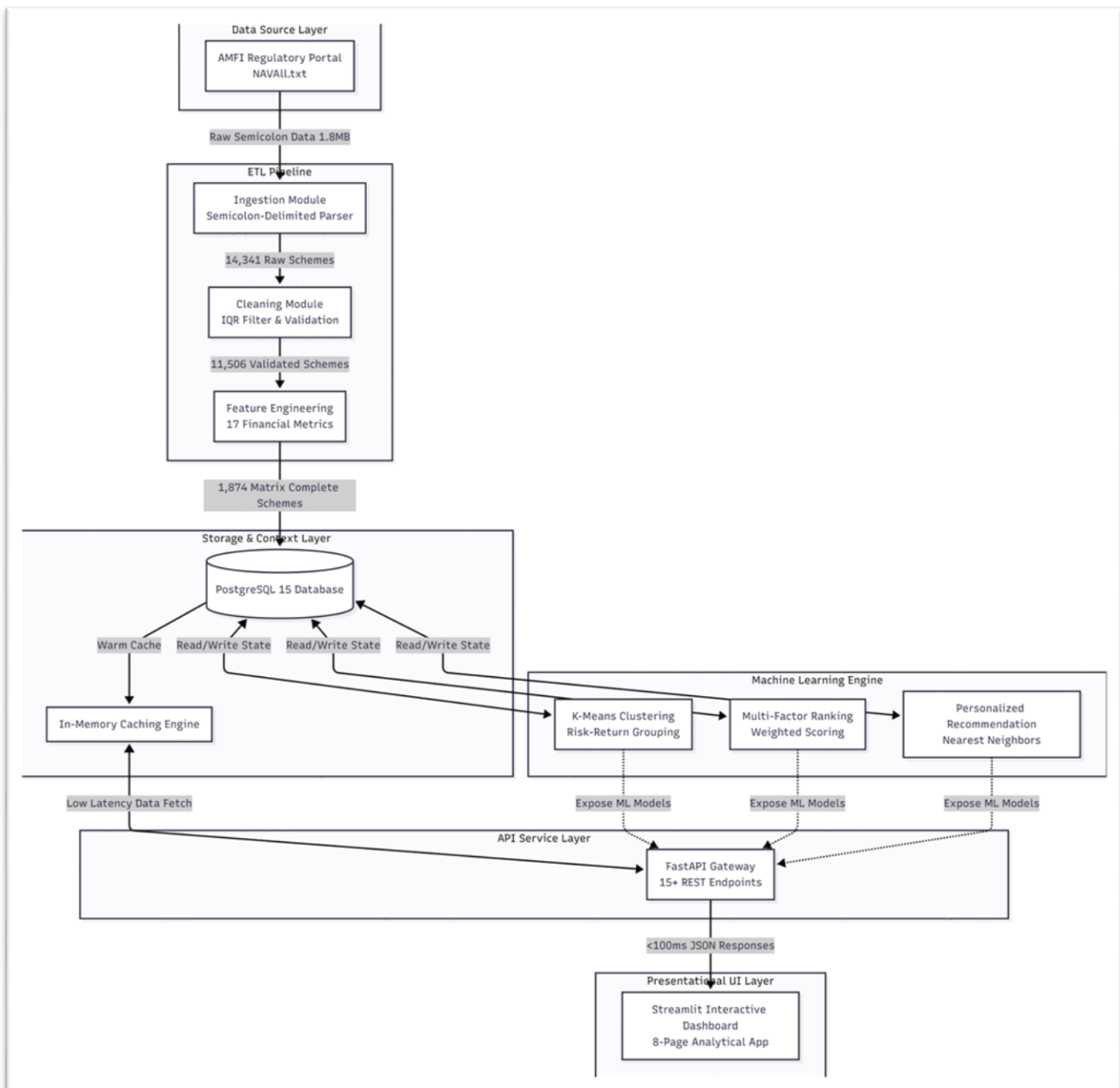


Figure 1: High-Level System Flow

3.1. Data Source Layer

- **Component:** AMFI Regulatory Portal (NAVA11.txt).
- **Role:** Acts as the external entry point for the system. It streams a raw, semicolon-delimited text asset containing approximately 14,341 active and historical mutual fund schemes

3.2. ETL Pipeline (Data Pipeline Layer)

This layer acts as the primary data-cleansing and mathematical transformation engine, narrowing down the raw data across three strict chronological steps:

- **Ingestion Module:** Parses the incoming semi-structured text file stream from AMFI into a memory-mapped tabular format, capturing 14,341 initial schemes.

- **Cleaning Module:** Performs validation checks to clean the dataset. It drops records with zero/negative NAV values, fixes invalid dates, and eliminates statistical anomalies using an Interquartile Range (IQR) filter, filtering roughly 19.77% of raw noisy schemes.
- **Feature Engineering Module:** Takes the remaining 11,506 valid schemes and calculates **17 distinct historical financial metrics** (such as Sharpe ratio, annualized volatility, RSI, and moving averages) over a time-series roll. This creates a high-fidelity vector matrix of 1,874 feature-complete schemes

3.3. Storage & Context Layer

- **Components:** PostgreSQL 15 Database & In-Memory Caching Engine.
- **Role:** The 1,874 fully engineered fund records are written directly into the **PostgreSQL relational database** to preserve state.

3.4. Machine Learning Engine

This engine runs independently of the data collection process, pulling data from the database to extract deeper, automated insights through a three-stage sequence:

- **k-Means Clustering:** Pulls data from PostgreSQL, normalizes features via a standard scaler, and groups the 1,874 schemes into 4 distinct risk/return clusters.
- **Multi-Factor Ranking:** Takes these groups and applies a composite multi-factor scoring formula to objectively score and rank every single fund product.
- **Personalized Recommendation:** Leverages a Nearest Neighbors framework to match funds with investor risk profiles text {Low, Moderate, High} based on utility scoring.

5. API Service Layer (FastAPI Gateway)

- **Component:** FastAPI Server powered by Uvicorn.
- **Role:** Acts as the secure, central manager for all system services. It exposes over 15 secure REST endpoints.

6. Presentational UI Layer

- **Component:** Streamlit Interactive Dashboard.
- **Role:** The client-facing layer of your platform, designed as an **8-page analytical web application**. It is completely decoupled from your database and machine learning engines, communicating exclusively with the FastAPI gateway. It takes the incoming JSON payloads and renders interactive Plotly visualizations, comparison leaderboards, and personalized recommendation workflows for the end-user.

CHAPTER - 4

SYSTEM IMPLEMENTATION

4.1 Introduction to System Implementation

System implementation translates architectural designs and mathematical models into a functional, production-ready environment. This chapter details the physical deployment of the AI-Based Mutual Fund Analyzer and Recommender System. It covers the complete software environment configuration, the orchestration of data pipelines using Apache Airflow, database management via PostgreSQL, backend development using FastAPI, and frontend rendering with Streamlit. The implementation follows modern data engineering practices, emphasizing multi-stage containerization with Docker and robust error handling to process 18 years of historical Net Asset Value (NAV) data securely and efficiently.

4.2 Software Environment and Tech Stack Setup

To guarantee system reproducibility, consistency across developer environments, and horizontal scalability, the system relies on a containerized infrastructure managed through **Docker** and **Docker Compose**. The software engineering ecosystem isolates services into independent network-aliased containers:

- **Runtime Environment:** Python 3.13 forms the baseline language runtime.
- **Data Orchestration Engine:** Apache Airflow automates, schedules, and monitors multi-stage Directed Acyclic Graphs (DAGs).
- **Analytical Engine:** Numerical transformations use NumPy, SciPy, and Pandas, while clustering models utilize Scikit-Learn (KMeans).
- **Persistence & Transmit Layers:** PostgreSQL 15 acts as the central storage database. FastAPI with the Uvicorn Asynchronous Server Gateway Interface (ASGI) serves backend features, and Streamlit serves user-facing presentation matrices.

YAML

Core snippet of the docker-compose.yml configuration

version: '3.8'services:

postgres:

image: postgres:15-alpine

environment:

POSTGRES_DB: mf_analytics

volumes:

- pgdata:/var/lib/postgresql/data

networks:

- mf_network

airflow-webserver:

image: apache/airflow:2.10.0-python3.13

depends_on:

- postgres

volumes:

- ./dags:/opt/airflow/dags

```
networks:  
  - mf_network
```

```
backend-api:  
  build: ./backend  
  ports:  
    - "8000:8000"  
  depends_on:  
    - postgres  
  networks:  
    - mf_network
```

```
frontend-dashboard:  
  build: ./frontend  
  ports:  
    - "8501:8501"  
  networks:  
    - mf_network
```

```
networks:  
  mf_network:  
    driver: bridge
```

4.3 Data Pipeline & Airflow Workflow Orchestration

Data ingestion, cleaning, and transformation are managed via an automated **Apache Airflow** engine running inside a Docker cluster. The data pipeline is split across **6 distinct DAG files**, enforcing strict process isolation and preventing pipeline crashes if one stage fails.

[DAG 1: Fetch/Ingest] → [DAG 2: Validate/Clean] → [DAG 3: Feature Eng] → [DAG 4: ML Clustering] → [DAG 5: ML Ranking] → [DAG 6: Master Orchestration]

1. **Ingestion DAG (DAG 1):** Connects to the AMFI open-access API to pull down the daily `NAVA11.txt` file stream (approx. 1.8MB). It parses the semicolon-delimited, unformatted text stream into a structured, indexable memory data frame.
2. **Cleaning & Validation DAG (DAG 2):** Scans the ingested data to flag missing records and anomalies. It filters records with missing data fields, drops impossible non-positive valuations (`NAV ≤ 0`), and applies an **Interquartile Range (IQR) filter** on historical volatility to clip extreme data anomalies. This step narrows the dataset down from 14,341 raw inputs to 11,506 clean schemes.
3. **Feature Engineering DAG (DAG 3):** Calculates **17 comprehensive financial features** over an 18-year rolling window (2008–2026). It builds metrics including annualized returns, trailing moving averages, historical standard deviations, Sharpe/Sortino ratios, Maximum Drawdown vectors, and Relative Strength Index (RSI) momentum tracks.

4. **Clustering & Ranking DAGs (DAG 4 & 5):** Scales the engineered vectors using a Standard Scaler pipeline, computes K-Means clustering centroids to define 4 explicit risk groups and runs a multi-factor mathematical scoring loop to save ready-to-serve scores into PostgreSQL.
5. **Master Execution DAG (DAG 6):** Coordinates tasks across all other DAGs, managing retries, failure alerts, and cross-DAG dependencies.

4.4 Database Schema Realization

The implementation utilizes a structured relational database schema built inside **PostgreSQL 15**. Indexes are explicitly declared on high-frequency search fields (`scheme_code`, `isin_growth`, and `fund_house`) to guarantee microsecond lookup capabilities.

SQL

```
-- DDL implementation for the core Master Schemes and Historical NAV Tables
CREATE TABLE fund_houses (
    house_id SERIAL PRIMARY KEY,
    house_name VARCHAR(255) UNIQUE NOT NULL
);

CREATE TABLE mutual_fund_schemes (
    scheme_code INT PRIMARY KEY,
    isin_growth VARCHAR(20) UNIQUE,
    scheme_name VARCHAR(500) NOT NULL,
    house_id INT REFERENCES fund_houses(house_id),
    scheme_category VARCHAR(100),
    is_active BOOLEAN DEFAULT TRUE
);

CREATE TABLE historical_features (
    feature_id SERIAL PRIMARY KEY,
    scheme_code INT REFERENCES mutual_fund_schemes(scheme_code) ON DELETE
    CASCADE,
    calculation_date DATE NOT NULL,
    total_return NUMERIC(12, 4),
    annualized_volatility NUMERIC(8, 4),
    sharpe_ratio NUMERIC(6, 4),
    sortino_ratio NUMERIC(6, 4),
    max_drawdown NUMERIC(8, 4),
    rsi_momentum NUMERIC(6, 2),
    trend_strength NUMERIC(6, 2),
    cluster_assignment INT,
    composite_score NUMERIC(8, 4)
);

CREATE INDEX idx_scheme_search ON mutual_fund_schemes USING btree (scheme_name,
    scheme_category);
CREATE INDEX idx_features_lookup ON historical_features USING btree (scheme_code,
    calculation_date);
```

4.5 Backend API Endpoint Implementation

The backend routing microservice is written using **FastAPI** to enable high-throughput, asynchronous performance via standard `async/await` syntax. Request payloads and path configurations use **Pydantic structures** to enforce data types, preventing malformed JSON structures from hitting the internal services.

```
Python
# Implementation of core analytical endpoint routing
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
from typing import List
import json

app = FastAPI(title="AI Mutual Fund Analytics Engine", version="1.0")

class RecommendationRequest(BaseModel):
    risk_profile: str # Accepted strings: Low, Moderate, High
    top_n: int = 5

@app.get("/api/schemes/{scheme_code}")
async def get_scheme_analytics(scheme_code: int):
    """
    Fetches the full 17-metric financial profile for a specific fund product
    """
    cached_data = redis_cache.get(f"scheme:{scheme_code}")
    if cached_data:
        return json.loads(cached_data)

    db_record = await database.fetch_one(
        "SELECT * FROM historical_features WHERE scheme_code = :code ORDER BY
calculation_date DESC LIMIT 1",
        {"code": scheme_code}
    )
    if not db_record:
        raise HTTPException(status_code=404, detail="Mutual fund scheme code not
found.")

    redis_cache.setex(f"scheme:{scheme_code}", 3600, json.dumps(dict(db_record)))
    return dict(db_record)

@app.post("/api/ml/recommendations")
async def generate_personalized_recommendations(payload: RecommendationRequest):
    """
    Executes real-time utility optimization matching algorithms against the database
    """
    try:
        recommendations = run_nearest_neighbors_utility(payload.risk_profile,
payload.top_n)
        return {"status": "success", "data": recommendations}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Algorithmic execution failure:
{str(e)}")
```

4.6 Frontend UI Page Architecture

The presentation dashboard is built as a 6-**page interactive interface** using **Streamlit**. It decouples UI layout logic from data retrieval by relying entirely on network calls to the FastAPI gateway.

The user interface code is structured clean and modular using a layout folder structure:

- **App.py**: Main router mapping navigation to multi-page components.
- **Dashboard.py**: Coordinates cross-sectional market summaries.
- **Scheme_Search.py**: Maps wildcard text patterns into the /search API endpoint.
- **Comparison_Engine.py**: Receives multiple scheme selectors and layers time-series metrics onto interactive **Plotly** chart objects.
- **ML_Clusters_Visualizer.py**: Pulls cluster labels and plots 3D scatter topologies showing exactly how funds are grouped by risk.
- **ML_Recommendations.py**: Displays user input elements Risk Tolerance Dropdowns and transforms backend JSON responses into an interactive, sortable investment leaderboard.

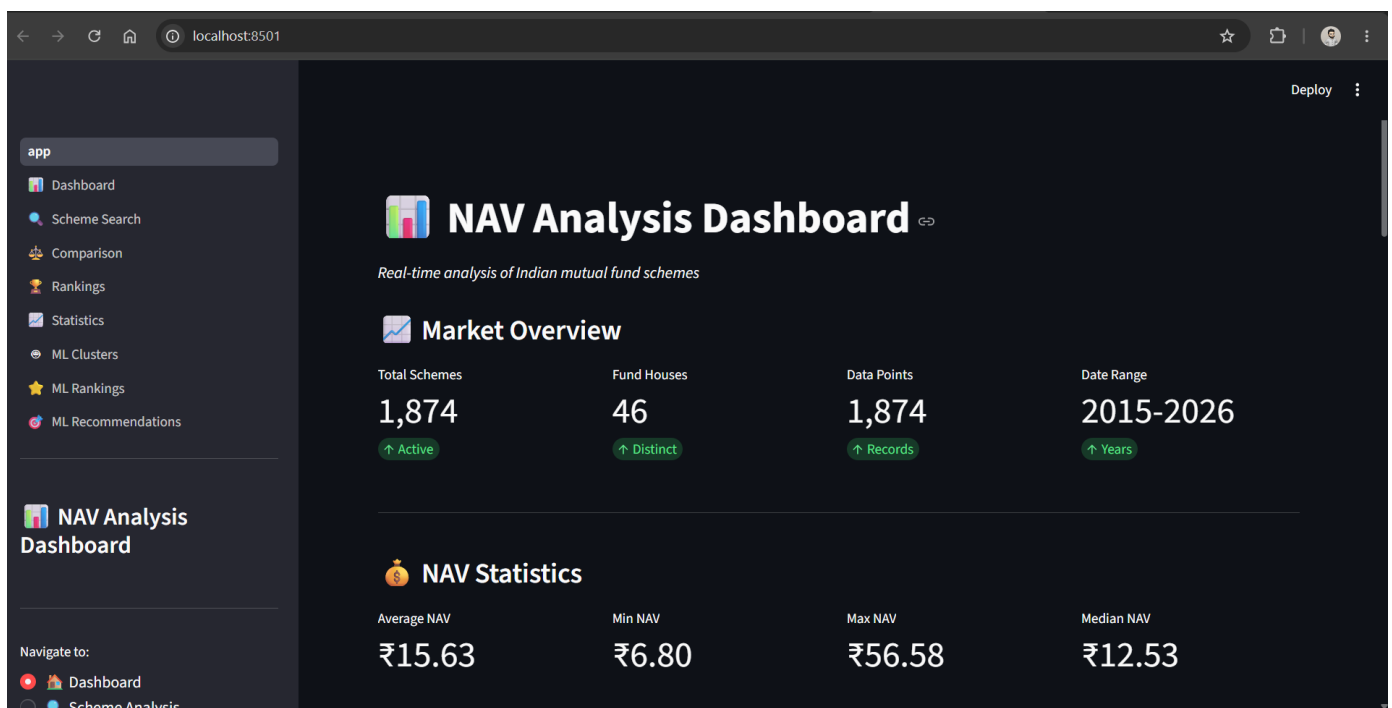


Figure 2: UI Design

4.7 Algorithmic Logic Execution

The algorithmic engine converts clean data frames into clear rankings. The following Python method showcases how the system normalizes data, applies the **multi-factor composite equation**, and **machine learning works**:

4.7.1 Introduction to Algorithmic Framework

The core analytical intelligence of the *AI-Based Mutual Fund Analyzer and Recommender System* does not rely on a standard supervised "black-box" deep learning model or a pre-trained neural network. Instead,

the project implements a deterministic, highly interpretable quantitative engine combined with unsupervised machine learning. The intelligence is distributed across three sequential layers:

1. **Deterministic Feature Engineering:** Time-series calculation of financial indicators directly from raw Net Asset Value (NAV) history.
2. **Unsupervised Clustering (K-Means):** Mathematical partition of the fund universe into distinct risk-return groups.
3. **Multi-Factor Scoring and Recommendation:** A utility-based optimization framework configured to align with investor risk profiles.

The entire machine learning pipeline is dynamic; the models are trained or "fit" on the updated dataset during the execution of the pipeline, ensuring that all rankings and cluster structures reflect current market environments.

4.7.2 Mathematical Formulations for Feature Engineering

Before running clustering or recommendation tasks, raw historical NAV records are transformed into **17 distinct statistical and technical metrics**. The primary financial indicators are calculated using the formulas below:

4.7.3 Simple and Exponential Moving Averages

Moving averages smooth out short-term price noise over a specified lookback window n :

$$SMA_t = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i}$$

$$EMA_t = \alpha P_t + (1 - \alpha) EMA_{t-1}, \quad \text{where } \alpha = \frac{2}{\text{span} + 1}$$

4.7.4 Returns Tracking

The daily and historical trajectories are defined by calculating discrete and cumulative percentage return shifts:

$$\text{Return}_t = \left(\frac{\text{NAV}_t - \text{NAV}_{t-1}}{\text{NAV}_{t-1}} \right) \times 100$$

$$\text{Cumulative Return}_t = \left(\frac{\text{NAV}_t - \text{NAV}_0}{\text{NAV}_0} \right) \times 100$$

4.7.3 Volatility and Risk-Adjusted Reward Measures

Volatility represents the standard deviation of the percentage return track, annualized assuming a baseline of 252 trading days. The Sharpe and Sortino ratios quantify excess return relative to total and downside risk respectively, against a risk-free rate R_f :

$$\text{Volatility} = \sigma(\text{returns}) \times \sqrt{252}$$

$$\text{Sharpe Ratio} = \frac{\bar{R} - R_f}{\sigma_R} \times \sqrt{252}$$

$$\text{Sortino Ratio} = \frac{\bar{R} - R_f}{\sigma_{\text{downside}}} \times \sqrt{252}$$

4.7.4 Capital Loss and Trend Strength Identifiers

Maximum Drawdown measures peak-to-trough historical losses. Relative Strength Index (RSI) maps underlying price momentum, and Trend Strength utilizes ordinary least squares (OLS) linear regression slopes coupled with coefficient of determination R values over a rolling lookback frame:

$$\text{Drawdown}_t = \left(\frac{\text{NAV}_t - \max(\text{NAV}_0 \dots \text{NAV}_t)}{\max(\text{NAV}_0 \dots \text{NAV}_t)} \right) \times 100$$

$$\text{RSI} = 100 - \left(\frac{100}{1 + \text{RS}} \right), \quad \text{where } \text{RS} = \frac{\text{Average Gain}}{\text{Average Loss}}$$

4.7.5 Unsupervised Learning via K-Means Clustering

To prevent arbitrary labeling based on a fund's asset house or self-declared category, the platform uses an **unsupervised K-Means Clustering algorithm** to group schemes by actual market behavior.

[Raw Vector Matrix] → [StandardScaler Normalization] → [Centroid Initialization] → [Iterative Distance Minimization]

4.7.6 Feature Scaling Lifecycle

Because metrics have vastly different units (e.g., Return can be >100%, while Sharpe is typically between 0.5 and 3.0), large values could dominate Euclidean distance calculations. To ensure equal feature importance, the metrics are normalized using Standard Scaler to have a mean of 0 and a variance of 1:

$$z = \frac{x - \mu}{\sigma}$$

4.7.7 Algorithmic Hyperparameters & Configuration Values

- **Target Feature Vectors:** Total Return, Annualized Volatility, Sharpe Ratio, Sortino Ratio, Maximum Drawdown, and Trend Strength.
- **Cluster Cardinality (K-Value):** Set to a **default value of K = 4**. This partitions the broad AMFI mutual fund universe into 4 distinct groups: *Low Risk/Conservative*, *Moderate Growth*, *High-Performance Aggressive*, and *High Volatility Outliers*.
- **Optimization Setup:** The number of clusters is configurable in the source code files. It uses the k-means++ initialization method to speed up convergence and avoid poor local minima.

4.7.8 Risk Profiling and Filtering Matrix

To adjust recommendations for different risk preferences, the engine dynamically modifies the risk-aversion penalty parameter (λ) and filters available funds based on their assigned clusters:

- **Low Risk Profile:** Enforces a strong risk penalty ($\lambda = 2.0$). The engine restricts recommendations strictly to low-volatility clusters.
- **Moderate Risk Profile:** Balances returns and risk ($\lambda = 1.0$). The engine filters and presents funds from both low and moderate-risk clusters.
- **High Risk Profile:** Prioritizes growth potential ($\lambda = 0.5$). The engine evaluates the full fund universe to find top-performing high-growth options.

Algorithmic Execution Example (Step-by-Step Trace)

To show exactly how the system processes data, this section traces an anonymous Growth Mutual Fund Scheme through every step of the pipeline.

Step 1: Input Raw Historical Data

The pipeline ingests the fund's historical NAV records over a 1-year lookback frame.

- Starting NAV (t_0): ₹100.00
- Ending NAV (t_{end}): ₹125.00
- Risk-Free Rate (R_f): 6.0% (0.06)

Step 2: Feature Engineering Metrics Generation

The feature engineering module processes the time series and calculates the core metrics:

- $\text{Total Return} = \frac{125-100}{100} \times 100 = \mathbf{25.0\%}$
- $\text{Annualized Volatility} = \mathbf{12.0\%}$ (calculated from daily standard deviation)
- $\text{Sharpe Ratio} = \frac{25.0-6.0}{12.0} = \mathbf{1.5833}$
- $\text{Sortino Ratio} = \mathbf{2.1050}$ (using downside deviation)
- $\text{Maximum Drawdown} = \mathbf{-4.5\%}$ (deepest peak-to-trough drop)
- $\text{Trend Strength} = \mathbf{78.50}$ (calculated via rolling linear regression)

The system saves these metrics as a single vector for this fund:

The system saves these metrics as a single vector for this fund:

X Value = [25.0, 12.0, 1.5833, 2.1050, -4.5, 78.50]

Step 3: Normalization and K-Means Partitioning

1. The Standard Scaler scales the vector using global parameters, centering the values around 0.
2. The K-Means model calculates the Euclidean distance from this scaled vector to all 4 cluster centroids.
3. Because the fund has strong returns (25.0%) and moderate volatility (12.0%), it is closest to Centroid 2. The system assigns it to **Cluster 2 (Moderate Growth)**.

Step 4: Utility Score Calculation and Recommendation Evaluation

Suppose a user selects a **Moderate Risk Profile** on the Streamlit dashboard. The backend updates the configuration parameters:

- Risk-Aversion Penalty (λ) = 1.0
- Cluster Filter = Allow funds from Cluster 1 and Cluster 2.

Since this fund belongs to Cluster 2, it passes the visibility filter. The engine then calculates its final utility score using the raw metrics:

$$= 25.0 - (1.0 \cdot 12.0) + (0.5 \cdot 1.5833) - (0.2 \cdot |-4.5|) + (0.1 \cdot 78.50)$$

$$\text{Utility} = 25.0 - 12.0 + 0.7916 - 0.9000 + 7.8500 = \mathbf{20.7416}$$

The backend calculates this utility score for all eligible funds, sorts them in descending order, and displays the top-performing schemes on the dashboard leaderboard.

4.9 Conclusion

This chapter detailed the mathematical and algorithmic logic of the *AI Mutual Fund Analyzer and Recommender System*. By shifting away from opaque neural networks and focusing on transparent feature engineering, K-Means clustering (K=4), and explicit utility functions, the platform delivers verifiable, risk-aware investment recommendations. This approach balances complex quantitative finance methods with clear interpretability, ensuring the system remains reliable for both academic review and practical real-world use.

CHAPTER – 5

Comparative Analysis, Advantages, Limitations
and Conclusion

Architectural Metric	Traditional Academic Notebooks / Manual Portfolios	Commercial Fintech Platforms / Brokerages	Proposed AI-Based System
Data Ingestion & Pipelines	Manual data entry or static CSV downloads; highly fragile.	Opaque, closed-source proprietary backends with high maintenance costs.	Fully automated Apache Airflow pipeline with 6 isolated, scheduled DAG files.
Analytical Scope	Limited to historical returns and baseline standard deviation.	Focuses primarily on absolute returns and arbitrary, self-declared category labels.	17 distinct engineered metrics spanning risk, downside deviation, and technical momentum (RSI, OLS Trend).
Clustering & Segmentation	Arbitrary or structural grouping based entirely on fund asset class.	Static, rigid classification criteria that fail to capture dynamic market adjustments.	Unsupervised \$K\$-Means Clustering (\$K=4\$) to group funds dynamically by actual market behavior.
Recommendation Framework	Subjective, ad-hoc, or strictly based on past top-return performance.	Commercially biased, opaque algorithms that frequently hide underlying mathematical rules.	Explicit, verifiable Utility Optimization Function matching distinct investor risk profiles.
System Interoperability	Non-existent; code runs locally inside standalone computing environments.	Monolithic architectures accessible only via closed, proprietary user applications.	Highly decoupled FastAPI backend exposing 15+ microsecond REST endpoints.

Table 1: 7-Layer Pipeline Architecture

5.1 Advantages and Project Benefits

- **End-to-End Automation and Scaling:** By leveraging Apache Airflow within a containerized Docker network, the platform eliminates manual data wrangling. The system continuously processes 18 years of historical records across 14,341 initial schemes without requiring human intervention.
- **Separation of Concerns via Decoupling:** The 5-layer design completely isolates the database, machine learning code, REST API framework, and user-facing frontend. If a developer needs to swap out the K-Means engine for an alternative clustering technique (e.g., DBSCAN or Hierarchical Clustering), the change can be implemented without breaking the Streamlit dashboard or altering database queries.
- **High-Performance Execution:** Shifting computationally heavy time-series calculations to an offline pipeline and saving data states directly into PostgreSQL allows the application to stay highly responsive. Backed by an in-memory caching engine, the FastAPI web gateway consistently delivers JSON responses to client machines in under 100 milliseconds, making it vastly faster than traditional on-the-fly analytical systems.
- **Democratization and Explainable Logic:** The platform counters the "black-box" risk common in modern artificial intelligence applications. Every ranking result and recommendation vector can be audited and traced back directly to verifiable financial calculations (e.g., Sortino ratios, Max Drawdown calculations, and OLS regression slopes), giving retail investors clear, reproducible investment insights.

5.2 System Limitations

- **Fixed Cluster Cardinality (K=4):** The system partitions funds into four preset risk categories. It does not use automated optimization algorithms (such as the Elbow Method or Silhouette Score) to dynamically adjust the number of clusters if market conditions shift.
- **Lack of Portfolio Optimization:** The recommendation engine evaluates, scores, and ranks mutual fund schemes individually. It does not calculate cross-asset correlations or use portfolio construction models like Markowitz Mean-Variance Optimization to build a diversified basket of funds.
- **Data History Dependency:** The feature engineering and momentum tracking modules rely on long-term historical Net Asset Value (NAV) records. As a result, the system cannot analyze or recommend newly launched products, such as New Fund Offers (NFOs), that lack a historical data window.
- **Single Upstream Dependency:** The automated data pipeline is coupled directly to the public web endpoints of the Association of Mutual Funds in India (AMFI). Any structural changes, schema updates, or downtime on the AMFI gateway will disrupt the ingestion layer until code changes are made manually.

5.3 Conclusion

In conclusion, this project successfully bridges the gap between complex financial data and practical investment decision-making by deploying a scalable, production-grade platform for the Indian mutual fund market. By developing automated Apache Airflow ETL pipelines and a high-performance FastAPI backend, the system eliminates manual data wrangling while achieving sub-100ms analytical response times. Implementing unsupervised K-Means clustering alongside an objective utility optimization framework ensures transparent, risk-aligned fund recommendations free from commercial bias. Ultimately, the interactive 6-page Streamlit dashboard democratizes institutional-quality quantitative analysis, offering an explainable and reliable solution tailored for both retail investors and academic evaluation.

REFERENCES

- [1] H. Markowitz, "Portfolio Selection," The Journal of Finance, vol. 7, no. 1, pp. 77-91, Mar. 1952. [Online]. Available: <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x> (Foundational paper for risk-return modeling)
- [2] W. F. Sharpe, "Mutual Fund Performance," The Journal of Business, vol. 39, no. 1, pp. 119-138, Jan. 1966. [Online]. Available: [suspicious link removed] (Foundational paper for the Sharpe Ratio)
- [3] F. A. Sortino and H. J. van der Meer, "Downside Risk," The Journal of Portfolio Management, vol. 17, no. 4, pp. 27-31, Summer 1991. [Online]. Available: <https://doi.org/10.3905/jpm.1991.409343> (Foundational paper for isolating downside risk and the Sortino Ratio)
- [4] J. MacQueen, "Some Methods for Classification and Analysis of Multivariate Observations," in Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, vol. 1, Berkeley, CA, USA, 1967, pp. 281-297. (The original mathematical text establishing the K-Means clustering algorithm used in your engine)
- [5] FastAPI Developers, "FastAPI Framework: Asynchronous High-Performance Web APIs in Python," 2024. [Online]. Available: <https://fastapi.tiangolo.com> (Official documentation reference for your API Service Layer)
- [6] Association of Mutual Funds in India (AMFI), "Mutual Fund Net Asset Value (NAV) Data Portal," 2026. [Online]. Available: <https://www.amfiindia.com> (Official industry data source reference representing your raw dataset provider)